

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings("ignore")

def generate_synthetic_data():
    """
    Generates a synthetic dataset resembling manufacturing product demand.
    Includes trend and seasonality.
    """
    np.random.seed(42)
    # Date range: 5 years of monthly data
    dates = pd.date_range(start='2019-01-01', periods=60, freq='M')

    # Base trend
    trend = np.linspace(100, 200, 60)

    # Seasonality (12-month cycle)
    seasonality = 20 * np.sin(np.linspace(0, 3.14 * 10, 60))

    # Noise (random fluctuations)
    noise = np.random.normal(loc=0, scale=10, size=60)

    # Total Demand
    demand = trend + seasonality + noise

    df = pd.DataFrame({'Date': dates, 'Demand': demand})
    df.set_index('Date', inplace=True)
    return df

def check_stationarity(timeseries):
    """
    Performs Augmented Dickey-Fuller test to check stationarity.
    """
    print("Results of Dickey-Fuller Test:")
    dfctest = adfuller(timeseries, autolag='AIC')
    dfoutput = pd.Series(dfctest[0:4], index=['Test Statistic', 'p-value', '#Lags Used', 'Number of Observations Used'])
    for key, value in dfctest[4].items():
        dfoutput['Critical Value (%s)' % key] = value
    print(dfoutput)
    return dfoutput['p-value']

def run_arima_analysis():
    # 1. Load Data
    print("--- Loading Data ---")
    data = generate_synthetic_data()
    print(data.head())

    # 2. Visualize Original Series
    plt.figure(figsize=(12, 6))
    plt.plot(data['Demand'], label='Observed Demand')
    plt.title('Monthly Manufacturing Product Demand (2019-2023)')
    plt.xlabel('Date')
    plt.ylabel('Units Sold')
    plt.legend()
    plt.savefig('original_series.png') # Save for report
    plt.show()

    # 3. Check Stationarity
    p_value = check_stationarity(data['Demand'])

    # 4. Differencing (if non-stationary)
    # We assume first-order differencing is needed for this synthetic trend data
    data_diff = data['Demand'].diff().dropna()

    plt.figure(figsize=(12, 6))
    plt.plot(data_diff, label='Differenced Demand', color='orange')

```

```

plt.title('Differenced Time Series (1st Order)')
plt.legend()
plt.savefig('differenced_series.png')
plt.show()

# Re-check stationarity on differenced data
check_stationarity(data_diff)

# 5. ACF and PACF Plots to determine p and q
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(data_diff, ax=ax1, lags=20)
plot_pacf(data_diff, ax=ax2, lags=20)
plt.tight_layout()
plt.savefig('acf_pacf.png')
plt.show()

# 6. Train-Test Split
train_size = int(len(data) * 0.8)
train, test = data['Demand'][0:train_size], data['Demand'][train_size:]

# 7. Fit ARIMA Model
# Order (p,d,q) chosen based on theoretical visual inspection of synthetic data
# p=1 (AR term), d=1 (Differencing), q=1 (MA term)
print("\n--- Fitting ARIMA(1,1,1) Model ---")
model = ARIMA(train, order=(1, 1, 1))
model_fit = model.fit()
print(model_fit.summary())

# 8. Forecasting
forecast_result = model_fit.get_forecast(steps=len(test))
forecast = forecast_result.predicted_mean
conf_int = forecast_result.conf_int()

# 9. Evaluation
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)
print(f'\nMean Squared Error: {mse:.2f}')
print(f'Root Mean Squared Error: {rmse:.2f}')
print(f'Mean Absolute Error: {mae:.2f}')

# 10. Visualization of Forecast
plt.figure(figsize=(14, 7))
plt.plot(train.index, train, label='Training Data')
plt.plot(test.index, test, label='Actual Demand', color='green')
plt.plot(test.index, forecast, label='ARIMA Forecast', color='red', linestyle='--')
plt.fill_between(test.index, conf_int.iloc[:, 0], conf_int.iloc[:, 1], color='pink', alpha=0.3, label='Confidence Interval')
plt.title('ARIMA Model Forecast vs Actuals')
plt.xlabel('Date')
plt.ylabel('Demand')
plt.legend()
plt.savefig('forecast_results.png')
plt.show()

if __name__ == "__main__":
    run_arima_analysis()

```

